

Reactive Agents with Drasi & Dapr

Jeffrey Zhang

Email: [REDACTED] — GitHub: JeffreyJPZ

1 Introduction

1.1 Motivation

Agentic AI is commonly used to predict and resolve problems before they occur, requiring access to data in near real-time. However, traditional patterns for reacting to data changes come with major disadvantages, exacerbating the challenge of building resilient, near real-time distributed systems. These patterns include polling, which is simple to implement but has a fundamental tradeoff between minimizing load on data sources and minimizing data staleness. Change data capture (CDC) avoids the aforementioned issues, but requires teams to build and maintain custom infrastructure. Using Drasi's ability to track derived entities (via Continuous Queries) and Dapr Agents' out-of-the-box durability for agents and agentic workflows, this proposed integration would help developers build scalable, event-driven AI systems in products and internal platforms. To accomplish this, Drasi would need a new "proxy" service that routes events to multiple publish-subscribe (pub/sub) topics. Dapr workflows and agents would consume from these topics via workflow decorator functions and agent middleware, supporting Drasi and providing the groundwork to integrate other backends.

1.2 Goals

This integration addresses two use cases for agentic AI and agentic workflow deployments:

1. A change is detected in an upstream data source (e.g. PostgreSQL, Redis, Azure EventHub). If the change satisfies some condition or affects some user-defined dynamic collection (see Drasi's "Solution Design" page for context), a specific workflow should be triggered with the change event data or affected entities.
2. Users interact with agents equipped with tools that can subscribe to data changes. Throughout the session, agents can receive notifications in the background and become "aware" of data changes without explicit prompting by the user.

In addition to these functional requirements, the integration should align with Dapr Agents' and Drasi's goals, including becoming production-ready (as of March 2026, Dapr Agents has not yet reached version 1.0 while Drasi is a CNCF Sandbox project). Therefore, the integration should expose intuitive interfaces to developers, leverage existing functionality and proven concepts when possible, and provide durability, resiliency, and observability.

1.3 Sample Scenario

To help "ground" our proposed approach, we will use the following (contrived) scenario:

Suppose we have an electronics retailer "Rainforest Electronics", whose inventory consists of various products such as laptops, RAM kits, and graphics cards. They are preparing to receive a new line of graphics cards, but due to quality control issues with previous lines, high GPU prices, reduced consumer spending, and supply chain shortages, they need to manage their inventory in a "smart" way, taking into account all of these factors when placing orders. Their current infrastructure includes:

- Managed databases for inventory, purchases, and product reviews
- A procurement software that integrates with most of the managed databases
- A market intelligence API providing competitor pricing data

With the integration, they should be able to use agentic workflows that integrate with their existing infrastructure to manage orders. Additionally, support staff should be able to monitor the launch with agents that can summarize reviews for "problematic" products.

1.4 Personal Information

1.4.1 Background

I am a third-year Computer Science student at the University of British Columbia, Canada, with a developed interest in system design through my Formula 1 play-by-play project. This proposal reflects the experience that I gained as an intern at a small startup, where I learned to understand systems not only by reading documentation (often vague), but also by tracing through codebases and researching patterns used by open-source projects. I became interested in Dapr while researching ways to make my project more flexible and maintainable as it grows, and my contribution to the Dapr runtime project (job metrics for the Dapr Jobs building block) provides me with a solid understanding of the Dapr ecosystem that both Drasi and Dapr Agents depend on. This includes its aims (improving developer productivity by abstracting away infrastructure concerns and providing intuitive, flexible APIs), and common contributor practices ("self-reviewing" PRs, justifying choices with multiple alternatives). Solving developer problems is also one of my passions. Through my involvement with OpenF1, an open-source Python API for Formula 1 data, I have enjoyed contributing not only through code (user-facing features and performance optimizations), but also by digging into long-standing issues and answering community questions.

1.4.2 Time Commitments

As this is a large (~ 350 hour) project, I can dedicate a minimum of 35-40 hours per week throughout the GSoC standard coding period to ensure that deliverables are completed on time. My timezone is UTC-07:00 (Pacific Time).

2 Related Work

A Drasi integration already exists for the LangChain framework (langchain-drasi), which currently exposes query discovery, subscriptions, and unsubscriptions from a single `DrasiTool` interface. The tool can either be directly invoked or used by agents, which addresses the agent subscription use case described in the "Goals" section. While the existing integration (built on MCP) serves as a good reference, the implementation is incompatible with the Dapr Agents framework for the following reasons:

- MCP is a stateful, session-based protocol (when used for notifications), with notifications being "pushed" by servers to clients over a long-lived connection. While this may be reasonable for the now-deprecated standalone agents, connections are subject to transient failures and are hard to scale without session affinity. To ensure durability and guaranteed delivery of notifications, the client and the server would need to support resumability and redelivery, and MCP session information would have to be externalized, adding unwanted complexity.
- While `DrasiTool` maintains internal state in the form of an MCP client, durable agents are backed by Dapr workflows, with tool calls as activities (atomic, isolated "functions"). Combined with the fact that activities are load-balanced by the Dapr placement service, session affinity would become a problem with this approach when scaling horizontally (related to the first reason).

Durable agents and Dapr in general already solve many of these problems through actors, workflows (which use actors), and asynchronous messaging. A new approach that leverages these patterns could be beneficial.

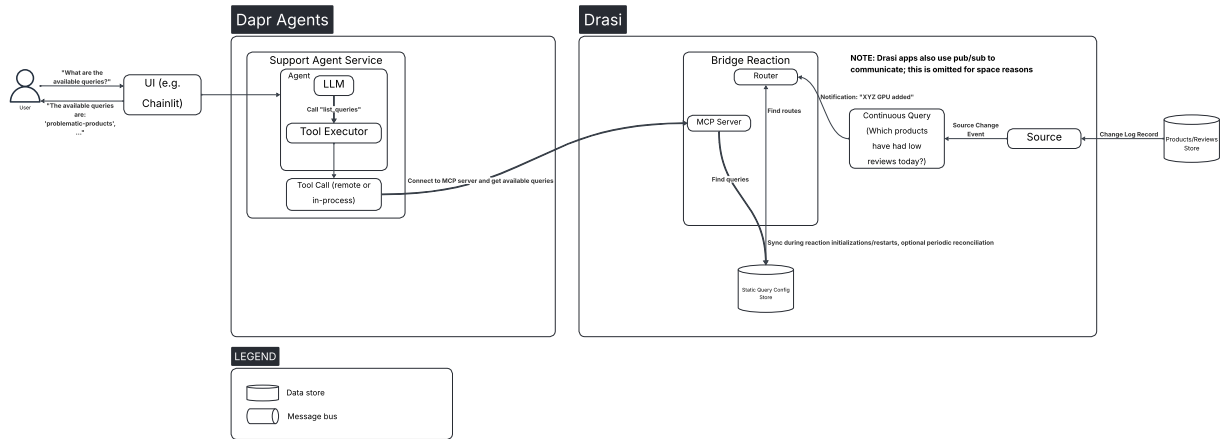
3 Proposed Approach

At a general level, the integration consists of two parts:

1. Dapr Agents functionality/infrastructure allowing agents to configure the Drasi Bridge reaction ("proxy" service that routes events to Dapr pub/sub topics) and to receive notifications.
2. The Drasi Bridge reaction, consisting of an event router, MCP server, and other necessary services/infrastructure.

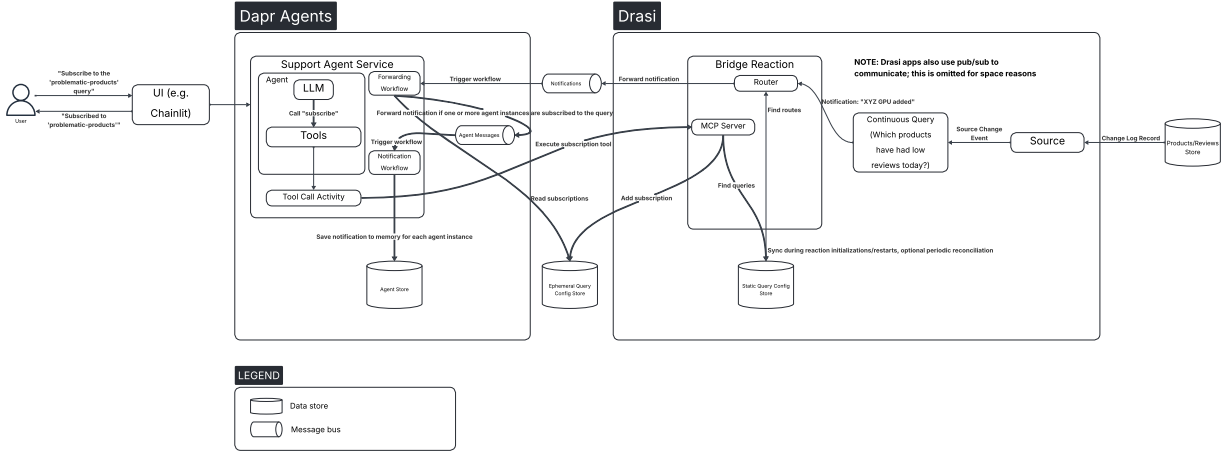
3.1 High-Level Overview

One possible architecture for the agent subscription use case is shown below:



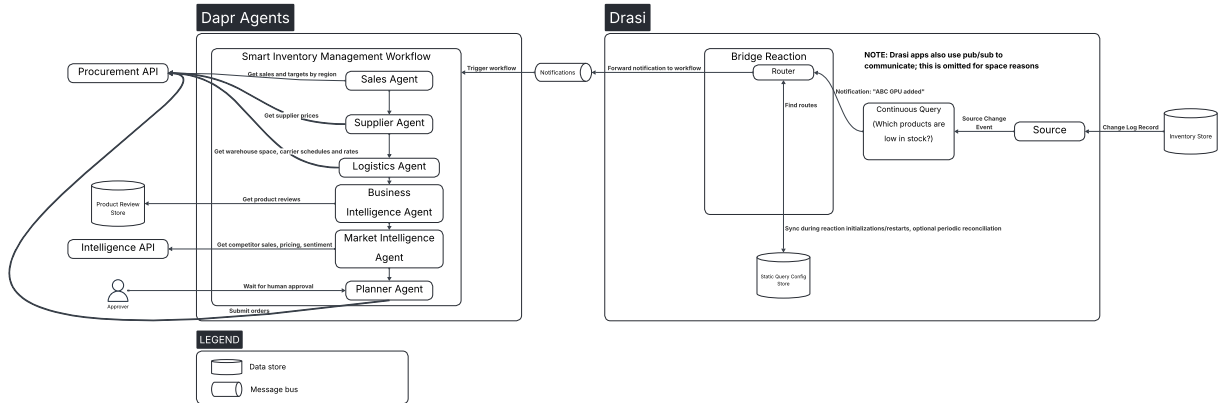
In the context of the example scenario, a user might ask a support agent for available queries (via HTTP, pub/sub, or console). The agent loop would process the input and eventually call the `list_queries` tool. The tool call would then connect to an MCP server that returns query IDs of available queries, including "problematic-products", a query tracking products with low reviews in a 24-hour window.

With the query name, the user can then ask the agent to subscribe to "problematic-products", so that it has up-to-date knowledge of such products in future conversations:



When the user asks the agent to subscribe, the flow is similar to query listing up until the tool call. With subscriptions, the tool call would mark a workflow instance ID (corresponding to a unique agent "session") as a subscriber to a specific Drasi query, saving the configuration to a state store. As products are added or removed from the "problematic-products" collection, the router inside the Bridge reaction would forward these events, eventually triggering a workflow that uses the subscription configuration to save notifications to memory. Unsubscriptions would go through a flow similar to subscriptions, removing the agent's workflow instance ID from the subscription configuration.

For the workflow use case, as events are forwarded to workflow topics, they should trigger workflow runs:



Using the example scenario, suppose we have an existing "smart" inventory management workflow that takes into account external conditions when placing orders (supplier pricing, market conditions, and more). When the product "XYZ GPU" is added to the collection of low-stock products, the router reads the query configuration and publishes the event to the relevant workflow topics, eventually triggering a workflow run that places orders for more "XYZ GPU"s.

3.2 Implementation and Justification

3.2.1 Dapr Agents Extension - Workflow Trigger

For the workflow trigger use case, most of the necessary functionality is already supported by Dapr and Dapr Agents. By default, Dapr pub/sub uses the CloudEvents 1.0 specification, and the existing `@message_router`

decorator uses streaming subscriptions to schedule workflows (users specify a pub/sub component name, pub/sub topic, and message models to validate the "data" field in the CloudEvents payload). The suggested `@drasi_trigger` decorator could take in the query ID and models for specific Drasi change types (insert, update, delete, control signal), wrapping the `@message_router` decorator. The following is a rough outline of the logic:

```

1  # Imports
2  ...
3
4  class DrasiQueryIDFilterMixin:
5      """Adds query ID filtering to DrasiEvent message models"""
6      _query_id: ClassVar[Optional[str]] = None # Optional class-level query ID to filter
7          ↳ by
8
9      @model_validator(mode="after")
10     def check_query_id(self):
11         if self.__class__._query_id is not None:
12             query_id = self.__class__._query_id or ""
13             event_query_id = self.root(...)
14
15             if event_query_id != query_id:
16                 raise ValueError(
17                     f"Mismatched query ID: expected {query_id}, got {event_query_id}"
18                 )
19
20     return self
21
22 # Another possible approach could be to allow a "post-validation" callback for
23 ↳ `@message_router` that takes in the validated model and arbitrary args (enabling
24 ↳ app-level filtering via
25 ↳ https://docs.pydantic.dev/latest/concepts/validators/#validation-context and
26 ↳ potentially semantic validation
27 ↳ https://github.com/pydantic/pydantic/issues/857#issuecomment-537570546), though this
28 ↳ may be too niche
29
30 def _model_with_validation(model: Any, query_id: str):
31     DrasiValidatedEvent = create_model(
32         "DrasiValidatedEvent",
33         __base__=(model, DrasiQueryIDFilterMixin),
34     )
35     DrasiValidatedEvent._query_id = query_id
36     return DrasiValidatedEvent
37
38 def drasi_trigger(
39     func: Optional[Callable[..., Any]] = None,
40     *,
41     pubsub: Optional[str] = None,
42     topic: Optional[str] = None,
43     dead_letter_topic: Optional[str] = None,
44     broadcast: bool = False,
45     message_model: Optional[Any] = None,
46     query_id: Optional[str] = None,
47 ) -> Callable[[Callable[..., Any]], Callable[..., Any]]:

```

```

40 # If query ID is given, filters events using a derived model that checks if query ID
   ↳ matches the event's query ID (ok since we only validate with the model?)
41 # Otherwise, the behaviour is the same as `@message_router`
42
43 def decorator(f: Callable[..., Any]) -> Callable[..., Any]:
44     if query_id:
45         # Pass through unchanged
46         return message_router(
47             func=f,
48             pubsub=pubsub,
49             topic=topic,
50             dead_letter_topic=dead_letter_topic,
51             broadcast=broadcast,
52             message_model=message_model
53         )
54
55     # Get concrete classes for models and perform other validation similar to the
   ↳ `@message_router` implementation
56     # Also check if models are Drasi events?
57     models = ...
58     validated_models = [_model_with_validation(m, query_id) for m in models]
59
60     return message_router(
61         func=f,
62         pubsub=pubsub,
63         topic=topic,
64         dead_letter_topic=dead_letter_topic,
65         broadcast=broadcast,
66         message_model=Union[tuple(validated_models)]
67     )
68
69 return decorator if func is None else decorator(func)

```

Message models would also need to be exposed. Drasi currently supports a "packed" and "unpacked" format for events, as well as "change" and "control" events. These models seem to be autogenerated, so we would need to ensure that `*Notification` models from `output-unpacked` schemas and `*Event` models from `query-output` schemas are exported. However, the Python Reaction SDK package already seems to export "packed" models, so we would need to update its corresponding Makefile to autogenerate "unpacked" models:

```

1 generate-types:
2     ...
3     npm run build ./output-unpacked --prefix ../../../../typespec
4     ...
5     poetry run datamodel-codegen --input-file-type jsonschema --input
   ↳ ../../../../typespec/output-unpacked/_generated/@typespec/json-schema
   ↳ --output-model-type pydantic_v2.BaseModel --use-annotated --output
   ↳ ./drasi_reaction/models

```

Considering how message models are validated in Dapr Agents, Pydantic models seem like the best choice to validate multiple event formats/schemas. The Dapr Agents interface could look something like:

```

1 from drasi.reaction.models.ChangeEvent import ChangeEvent
2 from drasi.reaction.models.ControlEvent import ControlEvent

```

```

3  from drasi.reaction.models.ChangeNotification import ChangeNotification
4  from drasi.reaction.models.ControlSignalNotification import ControlSignalNotification
5
6  # Other imports
7  ...
8
9  T = TypeVar("T")
10 S = TypeVar("S")
11
12 def _discriminate_change_event_format(data: Any) -> str:
13     # Discriminate between Drasi packed and unpacked formats for change events
14     # Packed events are discriminated on enums in _PackedOp
15     # Unpacked events are discriminated on enums in _UnpackedOp
16
17     if isinstance(data, dict):
18         if "kind" in data:
19             # Use result fields as packed events don't have an "op" field
20             if len(data.get("addedResults")) > 0:
21                 op = _PackedOp.ADD
22             elif len(data.get("updatedResults")) > 0:
23                 op = _PackedOp.UPDATE
24             elif len(data.get("deletedResults")) > 0:
25                 op = _PackedOp.DELETE
26         elif "op" in data:
27             # Unpacked event
28             op = data.get("op")
29     else:
30         if hasattr(data, "kind"):
31             if len(getattr(data, "addedResults", [])) > 0:
32                 op = _PackedOp.ADD
33             elif len(getattr(data, "updatedResults", [])) > 0:
34                 op = _PackedOp.UPDATE
35             elif len(getattr(data, "deletedResults", [])) > 0:
36                 op = _PackedOp.DELETE
37         elif hasattr(data, "op"):
38             op = data.get("op")
39
40     if op is None:
41         raise ValueError("Cannot discriminate Drasi change event")
42
43     return op
44
45 def _discriminate_event_format(data: Any) -> str:
46     # Discriminate Drasi packed events on "kind" and unpacked events on "op"
47     # Packed events are discriminated on enums in _Kind
48     # Unpacked events are discriminated on enums in _UnpackedOp
49
50     if isinstance(data, dict):
51         tag = data.get("kind") or data.get("op")
52     else:
53         tag = getattr(data, "kind", None) or get_attr(data, "op", None)
54

```

```

55     if tag is None:
56         raise ValueError("Cannot discriminate Drasi event")
57
58     return tag
59
60 class _Kind(StrEnum):
61     CHANGE = "change"
62     CONTROL = "control"
63
64 class _PackedOp(StrEnum):
65     # These must differ from unpacked ops for discrimination to work
66     ADD = "add"
67     UPDATE = "update"
68     DELETE = "delete"
69
70 class _UnpackedOp(StrEnum):
71     ADD = "i"
72     UPDATE = "u"
73     DELETE = "d"
74     CONTROL_SIGNAL = "x"
75
76 class DrasiBase(RootModel[T], Generic[T, S]):
77     """Wrapper model for Drasi event payloads, accessible through `.root` or `.data`.
78     ↳ Supports model serialization and deserialization."""
79     # Type adapter is necessary as there is no single "discriminator" field
80     _adapter: ClassVar[TypeAdapter[T]]
81     _event_type_adapter: ClassVar[TypeAdapter[S]]
82
83     @classmethod
84     def __pydantic_init_subclass__(cls, **kwargs):
85         super().__pydantic_init_subclass__(**kwargs)
86
87         metadata = getattr(cls, "__pydantic_generic_metadata__", None)
88         if not metadata:
89             return
90
91         args = metadata.get("args", ())
92         if not args:
93             return
94
95         event_type = args[0]
96
97         # guard against unresolved TypeVar
98         from typing import TypeVar as TV
99         if isinstance(S_concrete, TV):
100             return
101
102         cls._event_type_adapter = TypeAdapter(event_type)
103
104     @classmethod
105     def model_validate(cls, obj: Any, *args, **kwargs):
106         model = cls._adapter.validate_python(

```



```

106         obj,
107         strict=kwargs.get("strict"),
108         extra=kwargs.get("extra"),
109         from_attributes=kwargs.get("from_attributes"),
110         context=kwargs.get("context"),
111         by_alias=kwargs.get("by_alias"),
112         by_name=kwargs.get("by_name"),
113     )
114     return cls(root=model)
115
116     @classmethod
117     def model_validate_json(cls, data: str | bytes | bytearray, *args, **kwargs):
118         model = cls._adapter.validate_json(
119             data,
120             strict=kwargs.get("strict"),
121             extra=kwargs.get("extra"),
122             from_attributes=kwargs.get("from_attributes"),
123             context=kwargs.get("context"),
124             by_alias=kwargs.get("by_alias"),
125             by_name=kwargs.get("by_name"),
126         )
127         return cls(root=model)
128
129     @classmethod
130     def model_validate_strings(cls, obj: Any, *args, **kwargs):
131         model = cls._adapter.validate_strings(
132             obj,
133             strict=kwargs.get("strict"),
134             extra=kwargs.get("extra"),
135             from_attributes=kwargs.get("from_attributes"),
136             context=kwargs.get("context"),
137             by_alias=kwargs.get("by_alias"),
138             by_name=kwargs.get("by_name"),
139         )
140         return cls(root=model)
141
142     def model_dump(self, *args, **kwargs):
143         return self.__class__._adapter.dump_python(self.root, *args, **kwargs)
144
145     def model_dump_json(self, *args, **kwargs):
146         return self.__class__._adapter.dump_json(self.root, *args, **kwargs)
147
148     @property
149     def data(self) -> T:
150         return self.root
151
152 class DrasiAddEvent(DrasiBase[Union[ChangeEvent, ChangeNotification], S], Generic[S]):
153     _adapter = TypeAdapter(
154         Annotated[
155             Union[
156                 Annotated[ChangeEvent, Tag(_PackedOp.ADD)],
157                 Annotated[ChangeNotification, Tag(_UnpackedOp.ADD)]

```

```

158         ],
159         Discriminator(_discriminate_change_event_format)
160     ]
161 )
162
163 @model_validator(mode="after")
164 def validate_event(self):
165     # Validates the shape of events - if no concrete type is given (type adapter
166     ↪ doesn't exist), accepts all shapes
167     if self._event_type_adapter is not None:
168         if isinstance(self.root, ChangeEvent):
169             for evt in self.root.data.addedResults:
170                 self._event_type_adapter.validate_python(evt)
171         elif isinstance(self.root, ChangeNotification):
172             self._event_type_adapter.validate_python(self.root.payload.after)
173         else:
174             raise Exception(...)
175     return self
176
177 class DrasiUpdateEvent(DrasiBase[Union[ChangeEvent, ChangeNotification], S], Generic[S]):
178     _adapter = TypeAdapter(
179         Annotated[
180             Union[
181                 Annotated[ChangeEvent, Tag(_PackedOp.UPDATE)],
182                 Annotated[ChangeNotification, Tag(_UnpackedOp.UPDATE)]
183             ],
184             Discriminator(_discriminate_change_event_format)
185         ]
186     )
187
188 @model_validator(mode="after")
189 def validate_event(self):
190     # Validates the shape of events
191     if self._event_type_adapter is not None:
192         if isinstance(self.root, ChangeEvent):
193             for evt in self.root.data.updatedResults:
194                 self._event_type_adapter.validate_python(evt.before)
195                 self._event_type_adapter.validate_python(evt.after)
196         elif isinstance(self.root, ChangeNotification):
197             self._event_type_adapter.validate_python(self.root.payload.before)
198             self._event_type_adapter.validate_python(self.root.payload.after)
199         else:
200             raise Exception(...)
201     return self
202
203 class DrasiDeleteEvent(DrasiBase[Union[ChangeEvent, ChangeNotification], S], Generic[S]):
204     _adapter = TypeAdapter(
205         Annotated[
206             Union[
207                 Annotated[ChangeEvent, Tag(_PackedOp.DELETE)],
208                 Annotated[ChangeNotification, Tag(_UnpackedOp.DELETE)]

```

```

209         ],
210         Discriminator(_discriminate_change_event_format)
211     ]
212 )
213
214 @model_validator(mode="after")
215 def validate_event(self):
216     # Validates the shape of events
217     if self._event_type_adapter is not None:
218         if isinstance(self.root, ChangeEvent):
219             for evt in self.root.data.deletedResults:
220                 self._event_type_adapter.validate_python(evt)
221         elif isinstance(self.root, ChangeNotification):
222             self._event_type_adapter.validate_python(self.root.payload.after)
223         else:
224             raise Exception(...)
225     return self
226
227 class DrasiChangeEvent(DrasiBase[Union[ChangeEvent, ChangeNotification], S], Generic[S]):
228     _adapter = TypeAdapter(
229         Annotated[
230             Union[
231                 Annotated[ChangeEvent, Tag(_Kind.CHANGE)],
232                 Annotated[ChangeNotification, Tag(Literal[_UnpackedOp.ADD,
233                     ↪ _UnpackedOp.UPDATE, _UnpackedOp.DELETE])]]
233             ],
234             Discriminator(_discriminate_event_format)
235         ]
236     )
237
238 @model_validator(mode="after")
239 def validate_event(self):
240     # Validates the shape of events
241     if self._event_type_adapter is not None:
242         if isinstance(self.root, ChangeEvent):
243             for evt in self.root.data.addedResults:
244                 self._event_type_adapter.validate_python(evt)
245             for evt in self.root.data.updatedResults:
246                 self._event_type_adapter.validate_python(evt.before)
247                 self._event_type_adapter.validate_python(evt.after)
248             for evt in self.root.data.deletedResults:
249                 self._event_type_adapter.validate_python(evt)
250         elif isinstance(self.root, ChangeNotification):
251             if self.root.payload.before:
252                 self._event_type_adapter.validate_python(self.root.payload.before)
253                 self._event_type_adapter.validate_python(self.root.payload.after)
254             else:
255                 raise Exception(...)
256     return self
257
258 class DrasiControlEvent(DrasiBase[Union[ControlEvent, ControlSignalNotification]]):
259     _adapter = TypeAdapter(

```

```

260         Annotated[
261             Union[
262                 Annotated[ControlEvent, Tag(_Kind.CONTROL)],
263                 Annotated[ControlSignalNotification, Tag(_UnpackedOp.CONTROL_SIGNAL)]
264             ],
265             Discriminator(_discriminate_event_format)
266         ]
267     )
268
269     # This is equivalent to Union[DrasiChangeEvent, DrasiControlEvent]
270     class DrasiEvent(DrasiBase[Union[ChangeEvent, ChangeNotification, ControlEvent,
271         ↪ ControlSignalNotification]]):
272         _adapter = TypeAdapter(
273             Annotated[
274                 Union[
275                     Annotated[ChangeEvent, Tag(_Kind.CHANGE)],
276                     Annotated[ControlEvent, Tag(_Kind.CONTROL)],
277                     Annotated[ChangeNotification, Tag(Literal[_UnpackedOp.ADD,
278                         ↪ _UnpackedOp.UPDATE, _UnpackedOp.DELETE])],
279                     Annotated[ControlSignalNotification, Tag(_UnpackedOp.CONTROL_SIGNAL)]
280                 ],
281                 Discriminator(_discriminate_event_format)
282             ]
283         )

```

The above `*Event` models handle "packed" and "unpacked" Drasi event formats, making it easier for clients to write workflows that trigger on specific business conditions (any combination of add, update, delete, etc.). To demonstrate this, the smart inventory management workflow can be annotated as follows:

```

1     # Trigger workflow only when products are added to the low stock collection (no filtering
2     ↪ as workflows are triggered on every valid message)
3     # Equivalent to `@message_router(...)`
4     @drasi_trigger(
5         pubsub="pubsub", topic="inventory.low-stock", message_model=DrasiAddEvent[Product]
6     )
7     def inventory_workflow(ctx: DaprWorkflowContext, wf_input: dict) -> str:
8         ...
9
10    # Filter messages with a specific query ID before triggering workflows (useful if topics
11    ↪ can contain messages from multiple queries or services)
12    @drasi_trigger(
13        pubsub="pubsub", topic="inventory.alerts", message_model=DrasiAddEvent[Product],
14        ↪ query_id="inventory-low-stock"
15    )
16    def inventory_workflow(ctx: DaprWorkflowContext, wf_input: dict) -> str:
17        ...

```

3.2.2 Drasi Bridge Reaction - Workflow Trigger

To accompany the Dapr Agents workflow trigger extension, a router reaction that can forward Drasi events to multiple pub/sub topics would be beneficial, offering flexibility for different use cases (in contrast to the existing "DaprPostPubsub" reaction that only allows one topic per query). Since the Drasi Kubernetes controller performs rolling updates when the static configuration changes (via hashing), drift can be avoided

as long as the static configuration is synced when the app starts.

```
1  # Imports
2  ...
3
4  STATE_STORE_NAME = get_config_value("STATE_STORE_NAME") # Dapr state store instance name
5  KEY_PREFIX = get_config_value("KEY_PREFIX") # Optional state store key prefix
6  STATE_TTL_SECONDS = int(get_config_value("STATE_TTL_SECONDS")) # Optional state store key
   ↪ TTL
7  MESSAGE_TTL_SECONDS = int(get_config_value("MESSAGE_TTL_SECONDS")) # Optional individual
   ↪ pub/sub message TTL
8
9  def to_native_format(event: Union[ChangeEvent, ControlEvent] -> dict[str, Any]:
10     """Converts events into Drasi native "unpacked" format."""
11     ...
12
13  def build_payload(event: Union[ChangeEvent, ControlEvent], route: dict[str, Any]) ->
   ↪ dict[str, Any]:
14     return {
15         "pubsub_name": route["pubsubName"],
16         "topic_name": route["topicName"],
17         "data": to_native_format(event) if route["format"] == "Unpacked" else
   ↪     event.model_dump(),
18         "data_content_type": "application/json",
19         "metadata": {"cloudevent.type": ...} # Override type to support app-level filtering
20     }
21
22  async def publish_events_with_retry(payloads: list[dict[str, Any]]):
23     """Attempt to publish an event to multiple topics in parallel, with automatic retries
   ↪ on retrievable failures."""
24     msgs = [client.publish_event(**payload) for payload in payloads]
25
26     # Can safely assume event loop exists as it is managed by the reaction (which uses
   ↪ uvicorn)
27     results = await asyncio.gather(msgs, return_exceptions=True)
28
29     # Either keep trying for remaining topics or give up if best-effort is acceptable
30     ...
31
32     logger.info(f"publish success: ... publish failure: ...")
33
34  # Dapr client is accessible via closure
35  def change_event_handler_with_client(client: DaprClient):
36     async def change_event(event: ChangeEvent, query_configs: dict[Any, Any] | None):
37         # Assume query_configs is a list of {"pubsubName", "topicName", "format",
   ↪ "skipControlSignals"} objects
38
39         routes = query_configs.get(event.queryId)
40
41         if event.addedResults:
42             logger.info(f"Add event for query {event.queryId}")
43             payloads = [build_payload(event, route) for route in routes]
44             await publish_events_with_retry(payloads)
```

```

45         if event.deletedResults:
46             logger.info(f"Delete event for query {event.queryId}")
47             payloads = [build_payload(event, route) for route in routes]
48             await publish_events_with_retry(payloads)
49         if event.updatedResults:
50             logger.info(f"Update event for query {event.queryId}")
51             payloads = [build_payload(event, route) for route in routes]
52             await publish_events_with_retry(payloads)
53
54     return change_event
55
56 def control_event_handler_with_client(client: DaprClient):
57     async def control_event(
58         event: ControlEvent, query_configs: dict[Any, Any] | None = None
59     ):
60         logger.info(
61             f"Control signal {event.controlSignal} for query {event.queryId}"
62         )
63
64         routes = [
65             route for route in query_configs.get(event.queryId) if not
66             → route['skipControlSignals']
67         ]
68
69         payloads = [build_payload(event, route) for route in routes]
70         await publish_events_with_retry(payloads)
71
72     return control_event
73
74 def sync_static_query_config(client: DaprClient, reaction: DrasiReaction):
75     """Syncs the static query configuration with the state store."""
76     query_ids = [query_id for query_id in reaction.query_configs.keys()]
77
78     # Use Dapr GetBulkState method to get config for all query IDs
79     ...
80
81     # Replace static query configs in state store
82     ...
83
84     # We may also want to persist a list of all query IDs (for resource listing later on)
85     ...
86
87 if __name__ == "__main__":
88     # Hardcoded values can be set by environment variables
89     retry_policy = RetryPolicy(...)
90
91     with DaprClient(retry_policy=retry_policy) as client:
92         reaction = DrasiReaction(
93             on_change_event=change_event_handler_with_client(client),
94             on_control_event=control_event_handler_with_client(client),
95             parse_query_configs=parse_yaml_configs, # Built-in

```

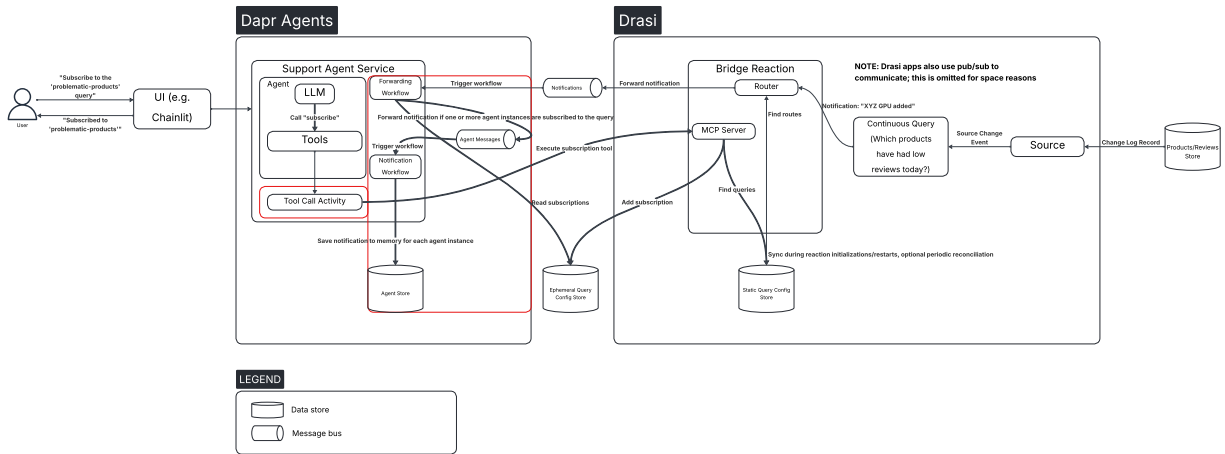
```

96     sync_static_query_config(client, reaction)
97
98     reaction.start()
99

```

3.2.3 Dapr Agents Extension - Agent Subscription

For the agent subscription use case, the main concern is the communication between Dapr Agents and Drasi (tool calling and notification handling) as well as query configuration persistence, highlighted in red:



For tool calling, Dapr Agents supports in-process tool execution as well as remote tool execution via MCP. Supporting both with a single implementation would be challenging (remote service calls would be needed for in-process tool execution, while remote tools can access server context depending on the framework). Since queries are owned by Drasi, query listing should be a capability that is exposed to other services, making remote tool execution the better choice:

```

1  # Using FastMCP, but this can be adapted for other frameworks
2
3  @mcp.tool
4  async def list_queries(ctx: dict[str, Any], context: Context = CurrentContext()) ->
5      dict[str, Any]:
6      """List available Drasi queries."""
7      return await ctx.list_resources()

```

Considering dynamic subscriptions and unsubscriptions, the query configuration can be split into two parts: a static configuration for workflows and other services (Drasi reaction specs, with queries mapped to Kubernetes ConfigMap objects), and an "ephemeral" configuration for agents (mapping of query IDs to workflow instance IDs and pub/sub components/topics). As for persistence, current best practices seem to prefer version control to manage static configuration (GitOps), but persisting static configuration in a state store could offer flexibility. This leads to several implementations, each with different advantages and drawbacks:

1. Share ephemeral query configuration state between agents and the Bridge reaction router. The agent service (runner) would update the ephemeral configuration in tool calls and read the configuration to save notifications. When the router receives a change event, it would read its static configuration and the ephemeral configuration (avoiding polling). However, the Dapr state management API only supports key-value pairs, meaning that frequent subscriptions and unsubscriptions from the same query by a large number of agents may result in contention. State schemas would also have to be shared between services, but "owned" by Dapr Agents since it writes the configuration.

2. Keep multiple copies of the ephemeral configuration, allowing subscriptions and unsubscriptions to dynamically configure the Bridge reaction's copy via MCP. To ensure eventual consistency, tool calls would need to retry multiple times if transient failures occur. As MCP is synchronous, this could make the agent appear "unresponsive". In addition, allowing agents to directly update the Bridge reaction without guardrails would increase its attack surface, as the static configuration could be accidentally or maliciously changed by an agent (if the same state store is used).
3. Keep multiple copies of the ephemeral configuration, but use a form of the outbox pattern to configure the Bridge reaction (transactions are probably not necessary, although the transactional variant is supported by Dapr). Activity retries could ensure eventual consistency while allowing the agent to "fire and forget", at the cost of additional complexity (the Bridge reaction would need a consumer to deduplicate messages).
4. Implement the architecture described in the "High-Level Overview" where ephemeral and static configurations are separated, and the router forwards notifications to statically configured topics (e.g. "inventory.alerts"). "Proxy" workflows would be responsible for reading the ephemeral configuration and forwarding each notification to agent topics. This extra indirection decouples the Drasi router from Dapr Agents as the router would not need to read from the ephemeral configuration, at the cost of unnecessary workflow runs/compute if queries do not have subscribed agents.

Considering the above options, the fourth approach is probably the cleanest, and enables agent subscriptions for not only Drasi but a variety of backends. Unnecessary workflow runs is less of an issue if topic granularity is reasonable and/or throughput is small (see this snippet on Drasi's limitations), and could be avoided completely if CloudEvents 'type' and event data is used to filter messages (this exists as routes for declarative and programmatic subscriptions but not for streaming subscriptions, pre and post-validation callbacks for `@message_router` executed around this line would actually be very useful).

With the fourth approach, subscription and unsubscription tools could be executed "in-process" (in activities) or remotely (on the Drasi MCP server). However, as subscriptions and unsubscriptions are for a specific query ID and benefit from query ID validation (in case of LLM hallucinations), they should be "owned" by the MCP server, making remote execution more suitable. This would require the MCP server to write to a shared store, but should not result in conflicts as the MCP server would be the only service that writes:

```

1  # Use "first-write-wins" for optimistic concurrency
2  state_options = StateOptions(concurrency=ConcurrencyMode.FIRST_WRITE)
3
4  @mcp.tool
5  async def subscribe(query_id: str, ctx: dict[str, Any], context: Context =
    ↳ CurrentContext()) -> dict[str, Any]:
6      """Subscribe to an available Drasi query matching the given query ID."""
7      # Get runtime dependencies from `ctx` injected by Dapr Agents
8      ...
9
10     state_metadata = {"ttlInSeconds": f"{ctx.get("ttl_seconds", "")}" if "ttl_seconds"
    ↳ in ctx else None
11     attempts = 10 # Hardcoded but could be set by configuration (through `ctx`?)
12
13     store_name = ctx.get("store_name", "")
14     key = f"{ctx.get("query_subscriptions_prefix", "")}:{query_id}" if
    ↳ "query_subscriptions_prefix" in ctx else query_id
15
16     # Check if query ID is valid using `list_resources`, short-circuiting if not
17     ...
18

```



```

19     # Since we are using optimistic concurrency control with etags, we may want a short
    ↪ retry loop to handle contention similar to 'save_state' in `DaprInfra`
20
21     # Always fetch latest query subscription state
22     # Assume Dapr client is initialized via FastMCP lifespan
23     client = ctx.lifespan_context["dapr_client"]
24     current = await client.get_state(
25         store_name=store_name,
26         key=key
27     ).json()
28     etag = current.get("etag", None)
29
30     # Initialize state if necessary (etag doesn't exist)
31     ...
32
33     # Short-circuit if this workflow instance ID is already subscribed or if the query ID
    ↪ is not valid, otherwise add this workflow instance ID
34     ...
35     state = {...}
36
37     for attempt in range(1, attempts + 1):
38         try:
39             await d.save_state(
40                 store_name=store_name,
41                 key=key,
42                 value=state,
43                 etag=etag,
44                 state_options=state_options,
45                 state_metadata=state_metadata,
46             )
47             return {...}
48         except Exception as exc:
49             if attempt == attempts:
50                 raise Exception(f"Failed to subscribe after {attempts} attempts") # Could
                    ↪ have a more specific exception
51             # Continue retrying
52             etag = None
53             time.sleep(min(0.25 * attempt, 1.0) * (1 + random.uniform(0, 0.25)))
54
55 @mcp.tool
56 async def unsubscribe(query_id: str, ctx: dict[str, Any], context: Context =
    ↪ CurrentContext()) -> dict[str, Any]:
57     """Unsubscribe from an available Drasi query matching the given query ID."""
58     # Same logic as `subscribe`
59     ...
60
61     # Short-circuit if this workflow instance ID is not subscribed or if the query ID is
    ↪ not valid, otherwise remove this workflow instance ID
62     ...
63     state = {...}
64
65     # Same logic as `subscribe`

```

Query configuration state could look like:

```

1  # Static configuration state
2  # We may need to maintain a separate list of query IDs to lookup every config (e.g. when
   ↳ listing MCP resources) which requires multiple network calls (assuming no in-memory
   ↳ cache). If the number of queries is small and writes are rare (likely the case), we
   ↳ could use a single entry with a list of all configs instead
3  "<key prefix>:<query ID>": {
4      "query_id": <query ID>, # May be redundant
5      "description": <description>, # Optional MCP resource description
6      "title": <title>, # Optional MCP resource title
7      "routes": [
8          {
9              "pubsubName": <pubsub name>,
10             "topicName": <topic name>,
11             "format": <format>,
12             "skipControlSignals": <skipControlSignals>,
13         },
14         ...
15     ]
16 }
17
18 # Ephemeral configuration state
19 # This key format allows cleanup of stale entries at the agent service level if state TTL
   ↳ is set
20 "<key prefix>:<query ID>:<Dapr-registered agent name>": {
21     "pubsub_name": <pubsub name>,
22     "topic_name": <topic name>,
23     "workflow_instance_ids": [<workflow instance ID>, ...],
24 }
```

Since Drasi and Dapr Agents already rely on Dapr's pub/sub capabilities for inter-service communication, notifications should use pub/sub to avoid long-lived connections (the already supported agent topic could be a good approach). To support application-level sessions, notifications should be saved to memory for each subscribed agent session (perhaps as formatted "user" messages following the Drasi LangChain integration). However, the only accepted way to interact with the durable agent loop/workflow is through external events, which pauses the loop. Given that workflows also need to be deterministic, a better approach may be to process notifications in a separate workflow (a similar pattern was used by the now-removed orchestrators). Workflows are registered with the workflow runtime (in the Dapr sidecar) when the agent starts, so the notification workflow would need to be "discovered" somehow.

Since the above logic is specific to Drasi, this could be achieved with a dedicated workflow app that saves notifications to memory. While this is probably OK for demo/proof-of-concept purposes, it requires a significant manual wiring (registering workflows and activities), adds overhead (deploying separate workflow apps), involves significant code duplication (saving to memory), and introduces lifetime coupling between workflow apps and agent apps. An alternative approach is to encapsulate notification handlers inside class-based agent middleware, which already exists in other agent frameworks such as LangChain, Microsoft Agent Framework, and Google ADK. Middleware logic could run before or after key actions in the agent loop including LLM calls and tool calls (an example is given in the [hyperref\[appendix:middleware\]](#)the appendix), allowing developers to customize agent behaviour (this is already on the Dapr Agents roadmap as indicated by this issue). This would also allow tool calls to receive a context object with runtime dependencies, as seen in the tool examples.

In LangChain, middleware capabilities are exposed as "hooks" or functions that execute before/after/around steps in the agent loop. In Dapr Agents, steps are implemented as workflow activities (or child workflows in the case of agents-as-tools). Therefore, hooks should serve as an entry point for custom workflows and activities to be executed. There are several potential approaches:

- Keep hooks and middleware logic as regular functions, which would require re-execution of an middleware chain if the agent fails at any point along the chain. This may be confusing in certain situations such as human-in-the-loop, where approval may need to be given multiple times. If there are many middleware layers, agents may also become "bloated" and encounter performance bottlenecks as middleware execution is in-process.
- Leverage child workflows for durability and resumability. Similar to agents-as-tools, this could allow for "multi-app workflow" systems where middleware execution is offloaded to middleware apps/specialized agents, enabling patterns such as centralized guardrails. Since child workflows can also spawn workflows, this could potentially facilitate more complex middleware patterns (DAGs) as well.
- A "hybrid" approach that allows hooks to be regular functions (called by the main agent workflow) with middleware logic as workflows, child workflows, and activities. This might be the most flexible approach, but developers would need to ensure that their logic does not block the agent workflow.

In any case, the agent would need to discover workflows and activities defined in the middleware and register them with the workflow runtime. The following code describes what a middleware "base" class could look like (using the second approach):

```

1  # Payload structure for middleware hooks
2  class WorkflowState(BaseModel, Generic[WorkflowMessage]):
3      instance_id: str = Field(
4          description="The calling workflow instance ID"
5      )
6      messages: List[WorkflowMessage] = Field(
7          default_factory=list, description="Messages exchanged during the workflow"
8      )
9      last_message: Optional[WorkflowMessage] = Field(
10         default=None, description="Last processed message in the workflow"
11     )
12     tool_call: Optional[Dict[str, Any]] = Field(
13         default=None, description="Tool call data (function name and arguments), defined
14         ↳ for `before_tool_call` hooks"
15     )
16     # Additional fields can be added
17     ...
18
19 def middleware_workflow(func: Optional[Callable[..., Any]] = None) -> Callable[..., Any]:
20     """Sets `_is_middleware_workflow = True` so the agent can later register this
21     ↳ workflow with the workflow runtime."""
22     setattr(func, "_is_middleware_workflow", True)
23     return func
24
25 def middleware_activity(func: Optional[Callable[..., Any]] = None) -> Callable[..., Any]:
26     """Sets `_is_middleware_activity = True` so the agent can later register this
27     ↳ activity with the workflow runtime."""
28     setattr(func, "_is_middleware_activity", True)
29     return func

```

```

28 class AgentMiddleware():
29     """
30     Base middleware class for supporting execution of custom logic (as child workflows)
    ↪ at key points in the main agent workflow loop. To implement custom middleware,
    ↪ extend this class and override the necessary hooks.
31     """
32
33     def __init__(
34         self,
35         *,
36         app_id: Optional[str] = None,
37         retry_policy: Optional[wf.RetryPolicy] = None,
38         instance_id_fn: Optional[Callable[wf.DaprWorkflowContext, str]] = None,
39     ):
40         """
41         Initialize the base agent middleware.
42
43         Args:
44             app_id: The Dapr app ID for cross-app routing. None for in-process.
45             retry_policy: Optional retry policy for workflow operations.
46             instance_id_fn: Optional callable that can be used to generate deterministic
    ↪ workflow instance IDs. Use this to help support idempotency/dedupe
    ↪ semantics for middleware logic.
47         """
48         self._app_id = app_id
49         self._retry_policy = retry_policy
50         self._instance_id_fn = instance_id_fn
51
52     def __post_init__(self, agent: Any):
53         """Logic to run after the agent is initialized but before the agent starts. If
    ↪ the callee uses agent state, it should create local copies to avoid race
    ↪ conditions."""
54         pass
55
56     @property
57     def name(self) -> str:
58         """Middleware instance name."""
59         # This affects the Dapr-registered names of any workflows/activities defined in
    ↪ the middleware
60         # May need to use the fully-qualified class name instead
61         return self.__class__.__name__
62
63     def before_tool_call(self, ctx: wf.DaprWorkflowContext, message: Dict[str, Any]):
64         """Hook to run before a tool call."""
65         # Return value conventions would differ from other frameworks since message is a
    ↪ copy of the input: we use the returned data whether it is modified or not,
    ↪ and None to short-circuit execution?
66         return message
67
68     # NOTE: depending on middleware semantics, lifecycle workflows/activities might
    ↪ belong to the agent runner/service moreso than the agent (they are not part of
    ↪ the agent loop), should they be defined here?

```

```

69     def on_shutdown(self, ctx: wf.DaprWorkflowContext, message: Dict[str, Any]):
70         """Hook to run when the agent runner shuts down. Use this when you want to clean
71         ↳ up session state (when reusing workflow instance IDs across multiple agent
72         ↳ workflow runs)."""
73         return message
74
75     # Other hooks (can omit from the project to minimize scope creep)
76     ...

```

We could then extend the base middleware class to add our tool call context injection and notification handler logic:

```

1  # Imports
2  ...
3
4  class DrasiMiddleware(AgentMiddleware):
5      def __init__(
6          self,
7          notifications_pubsub_name: str,
8          notifications_topic: str,
9          *,
10         message_models: Optional[Union[DrasiInsertEvent, DrasiUpdateEvent,
11         ↳ DrasiDeleteEvent, DrasiControlEvent, DrasiEvent]] = None,
12         on_notification: Optional[Callable[..., Any]] = None,
13         query_subscriptions_prefix: Optional[str] = None,
14         ttl_seconds: Optional[int] = None,
15     ):
16         """
17         Initializes the Drasi middleware.
18
19         Args:
20             notifications_pubsub_name: The Dapr pub/sub component name for subscription
21             ↳ notifications.
22             notifications_topic: The Dapr pub/sub topic name for subscription
23             ↳ notifications.
24             message_models: The message models used to validate subscription
25             ↳ notifications. Defaults to `DrasiEvent`.
26             on_notification: Optional handler called after a subscription notification is
27             ↳ validated against `message_models`. Overrides the default behaviour of
28             ↳ saving to memory if provided.
29             query_subscriptions_prefix: Optional key prefix for subscription state in the
30             ↳ shared Dapr state store. Defaults to "drasi-query-subscriptions".
31             ttl_seconds: Optional TTL for subscription state on a per-query basis.
32             ↳ Defaults to None (no expiration).
33         """
34
35         # DrasiMiddleware can't support cross-app routing since it relies on the agent
36         ↳ service via `@message_router` and `DaprInfra`
37         super().__init__()
38
39         if not isinstance(notifications_pubsub_name, str):
40             raise TypeError("DrasiMiddleware: notifications_pubsub_name must be a str")
41         if not isinstance(notifications_topic, str):

```

```

33         raise TypeError("DrasiMiddleware: notifications_topic must be a str")
34
35     # Only support one pub/sub name/topic for now due to `@message_router`
36     ↳ limitations
37     self._notifications_pubsub_name = notifications_pubsub_name
38     self._notifications_topic = notification_topic
39     self._models = models or DrasiChangeEvent
40     self._handler = handler or self.on_notification
41     self._query_subscriptions_prefix = query_subscriptions_prefix or
42     ↳ "drasi-query-subscriptions"
43     self._ttl_seconds = ttl_seconds
44
45 def __post_init__(self, agent: Any):
46     if agent.orchestrator:
47         raise RuntimeError("DrasiMiddleware: orchestrators are not supported")
48
49     # Copy infra to avoid race conditions with the main agent workflow
50     # `infra._state_default_model`, `infra._state_model`, `infra._last_etag` are
51     ↳ modified by some of the methods so a shallow copy may not be enough
52     self._infra = deepcopy(agent._infra)
53
54     # Use retry policy from agent if possible
55     self._retry_policy = agent._retry_policy or wf.RetryPolicy(...)
56
57     # Get name of the agent workflow as the agent's "name"
58     self._agent_name = agent.workflow_name
59
60     # Register message handlers with dynamically configured pub/sub names and topics
61     if self._infra is not None:
62         # Ensure notification handler is marked as a middleware workflow
63         if not hasattr(self._handler, "_is_middleware_workflow"):
64             middleware_workflow(self._handler)
65
66         # agent pub/sub name and topic are optional if wired via the agent runner,
67         ↳ this is just for clarity
68         message_router(
69             self._handler,
70             pubsub=self._infra.message_bus_name,
71             topic=self._infra.agent_topic_name,
72             message_model=self._models
73         )
74
75     # Workflows may trigger unnecessarily without app-level filtering
76     message_router(
77         self.process_and_forward_notification,
78         pubsub=self._notifications_pubsub_name,
79         topic=self._notifications_topic,
80         message_model=self._models
81     )
82
83     # Parts of the following code are adapted from the `DurableAgent` agent workflow

```

```

81 @middleware_workflow
82 def before_tool_call(self, ctx: wf.DaprWorkflowContext, message: Dict[str, Any]):
83     # Get main agent workflow instance ID
84     instance_id = message["instance_id"]
85
86     # Modify tool call args with Drasi-specific configuration
87     tool_call = message["tool_call"]
88     fn_name = tool_call["function"]["name"]
89
90     # Do nothing if tool call is not a Drasi tool
91     # Could also pass in tools and use their names instead
92     if fn_name not in ["list_queries", "subscribe", "unsubscribe"]:
93         return
94
95     raw_args = tool_call["function"].get("arguments", "")
96
97     try:
98         args = json.loads(raw_args) if raw_args else {}
99     except json.JSONDecodeError as exc:
100         raise AgentError(f"Invalid JSON in tool args: {exc}") from exc
101
102     # Add a `ctx` object as an tool call arg with:
103     # Dapr state store name
104     # Ephemeral query subscriptions prefix
105     # Dapr-registered agent name
106     # Workflow instance ID of this agent run/"conversation"
107     # Pubsub component name for the agent
108     # Pubsub topic name for the agent
109     # TTL for each query ID entry
110     ctx_obj = {
111         "store_name": self._infra.state_store.store_name, # Law of Demeter? Could
112         # → have a helper in `DaprInfra` to return the store name
113         "query_subscriptions_prefix": self._query_subscriptions_prefix,
114         "agent_name": self._agent_name,
115         "workflow_instance_id": instance_id,
116         "pubsub_name": self._infra.message_bus_name,
117         "topic": self._infra.agent_topic_name,
118         "ttl_seconds": self._ttl_seconds
119     }
120
121     args["ctx"] = ctx_obj
122
123     try:
124         new_args = json.dumps(args)
125     except (TypeError, ValueError) as exc:
126         raise AgentError(f"Unable to serialize tool args: {exc}") from exc
127
128     tool_call["function"]["arguments"] = new_args
129
130     return message
131 @middleware_workflow

```

```

132 def on_shutdown(self, ctx: wf.DaprWorkflowContext, message: Dict[str, Any]):
133     # Unsubscribe this workflow instance from all queries.
134     yield ctx.call_activity(
135         self.unsubscribe,
136         input={
137             "instance_id": message["instance_id"]
138         },
139         retry_policy=self._retry_policy,
140     )
141
142 @middleware_activity
143 def unsubscribe(self, ctx: wf.WorkflowActivityContext, payload: Dict[str, Any]):
144     # Unsubscription logic is similar to the `unsubscribe` tool, but we need to
145     ↪ unsubscribe from all Drasi query IDs (do we need to get all queries first
146     ↪ similar to the `list_queries` tool?)
147     ...
148
149 # NOTE: similar to `on_shutdown`, notification workflows/activities are not part of
150 ↪ the agent loop, they could instead be part of another object (e.g.
151 ↪ `DrasiQueryRoutes`) that is passed to `AgentRunner.register_routes` (would need
152 ↪ to support additional targets for AgentRunner's overridden `register_routes`
153 ↪ method), this would scatter configuration and make saving notifications to memory
154 ↪ harder though
155
156 @middleware_workflow
157 def on_notification(self, ctx: wf.DaprWorkflowContext, message: Dict[str, any]):
158     """Default notification handler that saves Drasi notifications to memory.
159     ↪ @message_router decorated workflows are scheduled by the agent
160     ↪ runner/service, so this must save notifications for all currently subscribed
161     ↪ agents under the agent service."""
162
163     # Dapr by default uses the app ID as the consumer group name for pub/sub, and if
164     ↪ each agent app is associated with at most one unique (pubsub_name,
165     ↪ topic_name) pair (through `AgentPubsubConfig`), we don't need any deduping
166     ↪ logic
167
168     # `@message_router` puts CloudEvents metadata under the `_message_metadata`
169     ↪ attribute
170     metadata = message.get("_message_metadata", {}) or {}
171     source = message.get("source", "")
172     pubsub_name = message.get("pubsubname", "")
173     topic = message.get("topic", "")
174
175     logger.info("on_notification: received notification from %s from topic %s on
176     ↪ pubsub %s", source, topic_name, pubsub_name)
177
178     # Get query ID from packed or unpacked event
179     query_id: str = message.get("queryId", "") or message.get("payload",
180     ↪ {}).get("source", {}).get("queryId", "")
181
182     instance_ids = yield ctx.call_activity(
183         self.get_subscribed_instance_ids,
184         input={

```



```

168         "query_id": query_id
169         "pubsub_name": pubsub_name
170         "topic_name": topic_name
171     },
172     retry_policy=self._retry_policy,
173 )
174
175     # Save notifications in parallel
176     all_tasks: List[Any] = []
177     for instance_id in instance_ids:
178         all_tasks.append(
179             ctx.call_activity(
180                 self.save_notification,
181                 input={
182                     "instance_id": instance_id,
183                     "notification": message,
184                 },
185                 retry_policy=self._retry_policy,
186             )
187         )
188
189     yield wf.when_all(all_tasks)
190
191 @middleware_workflow
192 def process_and_forward_notification(self, ctx: wf.DaprWorkflowContext, message:
193     ↪ Dict[str, any]):
194     """Internal "proxy" workflow that processes Drasi notifications from a known
195     ↪ topic. If there is at least one subscriber to this Drasi query via the agent
196     ↪ topic (pubsub_name, topic), forwards the notification to the agent topic."""
197
198     # Drasi also uses CloudEvents
199     metadata = message.get("_message_metadata", {}) or {}
200     source = metadata.get("source", "")
201     pubsub_name = message.get("pubsubname", "")
202     topic = message.get("topic", "")
203
204     logger.info("process_and_forward_notification: received notification from %s from
205     ↪ topic %s on pubsub %s", source, topic_name, pubsub_name)
206
207     query_id: str = message.get("queryId", "") or message.get("payload",
208     ↪ {}).get("source", {}).get("queryId", "")
209
210     # Get workflow instance IDs of subscribed agents
211     instance_ids = yield ctx.call_activity(
212         self.get_subscribed_instance_ids,
213         input={
214             "query_id": query_id,
215             "pubsub_name": self._infra.message_bus_name,
216             "topic": self._infra.agent_topic_name
217         },
218         retry_policy=self._retry_policy,
219     )

```

```

215
216     if len(instance_ids) > 0:
217         # Forward notification to agent topic
218         yield ctx.call_activity(
219             self.forward_notification_to_agent_topic,
220             input={
221                 "query_id": query_id,
222                 "pubsub_name": self._infra.message_bus_name,
223                 "topic": self._infra.agent_topic_name
224             },
225             retry_policy=self._retry_policy,
226         )
227
228     @middleware_activity
229     def get_subscribed_instance_ids(self, ctx: wf.WorkflowActivityContext, payload:
230         ↳ Dict[str, Any]) -> list[str]:
231         """Get workflow instance IDs of agents that are subscribed to a Drasi query via the
232         ↳ agent topic (pubsub_name, topic)"""
233         query_id = payload.get("query_id", "")
234         pubsub_name = payload.get("pubsub_name", "")
235         topic = payload.get("topic", "")
236
237         # Use the `StateStoreService` provided by `DaprInfra` (assumes state store is
238         ↳ configured on the agent)
239         subscriptions = self._infra.state_store.load(
240             key=self._query_subscriptions_key
241         ).json()
242
243         # Get workflow instance IDs by filtering on (pubsub_name, topic)
244         workflow_instance_ids = ...
245
246         return workflow_instance_ids
247
248     @middleware_activity
249     def save_notification(self, ctx: wf.WorkflowActivityContext, payload: Dict[str,
250         ↳ Any]):
251         """Saves Drasi notifications to memory for a given agent workflow instance ID."""
252         instance_id: str = payload.get("instance_id", "")
253         notification: dict[str, Any] = payload.get("notification", {})
254
255         serialized_notification: str = json.dumps(notification)
256
257         # Ensure we have the latest workflow state
258         entry = self._infra.get_state(instance_id)
259
260         # Format user message and update the workflow entry
261         user_message_model = UserMessage(
262             content=f"Use the following notification to inform future responses:
263                 ↳ {serialized_notification}"
264             role="user"
265         )

```

```

262     entry.messages.append(user_message_model)
263     if hasattr(entry, "last_message"):
264         entry.last_message = user_message_model
265
266     self._infra.save_state(instance_id)
267
268     @middleware_activity
269     def forward_notification_to_agent_topic(self, ctx: wf.WorkflowActivityContext,
270     ↪ payload: Dict[str, Any]):
271         # Publish to agent topic
272         ...

```

We would also need to modify the `DurableAgent` class (to run middleware hooks), the pub/sub wiring methods for the agent service `AgentRunner` (to discover workflow and activity handlers in middleware), and the runner's lifecycle methods (to run middleware lifecycle hooks). The following code assumes that the first defined middleware has the highest precedence (its `before_*` methods would be called first, and its `after_*` methods would be called last):

```

1  # `dapr_agents/agents/base.py`
2  def __init__(...):
3      ...
4      self.middleware: List[Any] = list(middleware or [])
5
6      # Call any post initialization hooks
7      for m in self.middleware:
8          if hasattr(m, "__post_init__"):
9              m.__post_init__(agent=self)
10
11 # `dapr_agents/agents/durable.py`
12 def agent_workflow(self, ctx: wf.DaprWorkflowContext, message: dict):
13     ...
14     # Only consider regular, non-agent tools for now
15     for idx, tc in enumerate(tool_calls):
16         ...
17         # Invoke and execute regular tools.
18         else:
19             # Run `before_tool_call` hooks
20             # A "cleaner" approach could be similar to
21             # https://github.com/victordibia/designing-multiagent-systems/blob/1aebb3111
22             ↪ c667238e31fe3c698af2dc49baf4f3f/picoagents/src/picoagents/_middleware.py
23             ↪ #L159-L486, but yielding executors that wrap workflow tasks instead,
24             ↪ allowing us to control execution and checkpoint progress from the loop
25             ↪ (func can be None for non-wrap hooks e.g. before/after tool calls?)
26             tool_call: dict[str, Any]
27             if len(self.middleware) > 0:
28                 tool_call = deepcopy(tc)
29                 payload = WorkflowState(instance_id=ctx.instance_id,
30                 ↪ messages=entry.messages, last_message=entry.last_message,
31                 ↪ tool_call=tool_call)
32
33             # This executes the middleware chain on one tool at a time, should this
34             ↪ also be parallelized across tools?
35             for m in self.middleware:

```

```

29         if isinstance(m, AgentMiddleware) and hasattr(m, "before_tool_call"):
30             if hasattr(m, "_instance_id_fn"):
31                 id = m._instance_id_fn(ctx)
32             else:
33                 id = str(uuid.uuid4())
34
35         result = yield ctx.call_child_workflow(
36             workflow=middleware_workflow_id(self.name, m.name,
37                 ↪ m.after_agent),
38             input=payload.model_dump(),
39             instance_id=id,
40             app_id=m._app_id,
41             retry_policy=m._retry_policy or self._retry_policy,
42         )
43         # Update payload with result and save state
44         ...
45
46     # Validate tool_call
47     ...
48
49     activity_tasks.append(
50         ctx.call_activity(
51             self.run_tool,
52             input={
53                 "tool_call": tool_call if tool_call is not None else tc,
54                 "instance_id": ctx.instance_id,
55                 "time": ctx.current_utc_datetime.isoformat(),
56                 "order": idx,
57             },
58             retry_policy=self._retry_policy,
59         )
60     )
61
62     ...
63
64     # Tool results are trickier since tools can be executed sequentially or in
65     ↪ parallel, this can be discussed later
66
67     ...
68
69     # Run `after_agent` hooks
70     if len(self.middleware) > 0:
71         payload = WorkflowState(instance_id=ctx.instance_id, messages=entry.messages,
72             ↪ last_message=entry.last_message)
73         for m in self.middleware:
74             if isinstance(m, AgentMiddleware) and hasattr(m, "after_agent"):
75                 if hasattr(m, "_instance_id_fn"):
76                     id = m._instance_id_fn(ctx)
77                 else:
78                     id = str(uuid.uuid4())
79
80             result = yield ctx.call_child_workflow(
81                 workflow=middleware_workflow_id(self.name, m.name, m.after_agent),
82                 input=payload.model_dump(),

```

```

78         instance_id=id,
79         app_id=m._app_id,
80         retry_policy=m._retry_policy or self._retry_policy,
81     )
82     # Update payload with result and save state
83     ...
84
85 def register_workflows(self, runtime: wf.WorkflowRuntime) -> None:
86     ...
87     # Register workflows and activities in order of middleware declaration
88     # Assume there exists `middleware_workflow_id`/`middleware_activity_id` methods that
89     ↪ take in an agent name, middleware name, and method name and returns a
90     ↪ (normalized) Dapr-registered workflow/activity name e.g.
91     ↪ 'dapr.agents.{agent_name}.middleware.{middleware_name}.{method}.{workflow |
92     ↪ activity}' (matching existing `agent_workflow_id`, `broadcast_workflow_id`,
93     ↪ `orchestration_workflow_id` helpers)
94 for m in self.middleware:
95     for _, wf in get_decorated_methods(m, "_is_middleware_workflow"):
96         runtime.register_workflow(self._named(wf, middleware_workflow_id(self.name,
97             ↪ m.name, wf)))
98     for _, activity in get_decorated_methods(m, "_is_middleware_activity"):
99         runtime.register_activity(self._named(activity,
100             ↪ middleware_activity_id(self.name, m.name, activity)))
101
102     ...
103
104 # `dapr_agents/workflow/runners/agent.py`
105 def _build_pubsub_specs(
106     self, agent: DurableAgent, config: Any
107 ) -> list[PubSubRouteSpec]:
108     handlers = get_decorated_methods(agent, "_is_message_handler")
109     if not handlers:
110         return []
111
112     # Find all pubsub message handlers on middleware
113 for m in agent.middleware:
114     middleware_handlers = get_decorated_methods(m, "_is_message_handler")
115     if not middleware_handlers:
116         continue
117
118     # get_decorated_methods returns a dict with keys as function `__name__`'s, so we
119     ↪ should prefix them with the middleware's name in case of conflicts
120 middleware_handlers = {f"middleware_{m.name}_{k}": v for k, v in
121     ↪ middleware_handlers.items()}
122
123     handlers.update(middleware_handlers)
124
125     ...
126
127 # Use `key` to differentiate between agent and middleware workflows
128 for key, handler in handlers.items():
129     meta = getattr(handler, "_message_router_data", {})
130     is_broadcast = meta.get("is_broadcast", False)
131
132     ...

```

```

121     registered_name: Optional[str]
122     pubsub_name: Optional[str]
123     topic: Optional[str]
124
125     if "middleware" in key:
126         _, name, handler = key.split("_")
127         registered_name = middleware_workflow_id(agent.name, name, handler)
128         # Fallback to agent config if necessary
129         pubsub_name = meta.get("pubsub", None) or config.pubsub_name
130         topic = meta.get("topic", None)
131         if topic is None:
132             topic = config.broadcast_topic if is_broadcast else config.agent_topic
133     else:
134         # Get agent/broadcast workflow name as normal
135         registered_name = ...
136         pubsub_name = config.pubsub_name
137         topic = config.broadcast_topic if is_broadcast else config.agent_topic
138     ...
139     specs.append(
140         PubSubRouteSpec(
141             pubsub_name=pubsub_name,
142             topic=topic,
143             handler_fn=named_handler,
144             message_model=message_model,
145         )
146     )
147
148 def shutdown(self, agent: Optional[DurableAgent] = None) -> None:
149     """
150     Unwire subscriptions and close owned clients.
151
152     Args:
153         agent: Durable agent instance.
154
155     Returns:
156         None
157     """
158     if agent:
159         # We need to shutdown a single agent
160         # First verify we're managing it
161         with self._lock:
162             if agent in self._managed_agents:
163                 ...
164                 # Call middleware shutdown hooks before the agent (potentially) shuts
165                 ↳ down the workflow runtime
166                 for m in agent.middleware:
167                     if isinstance(m, AgentMiddleware) and hasattr(m, "on_shutdown"):
168                         if hasattr(m, "_instance_id_fn"):
169                             id = m._instance_id_fn(ctx)
170                         else:
171                             id = str(uuid.uuid4())

```

```

172         await self.run_workflow_async(
173             m.on_shutdown,
174             {}, # No payload necessary
175             instance_id=id,
176         )
177     agent.stop()
178     ...
179     ...
180     ...

```

The agent could then be instantiated as follows (the following code is adapted from the Dapr Agents Chainlit Data Agent demo):

```

1  # Imports, other definitions and configuration
2  ...
3
4  _agent: Optional[DurableAgent] = None
5  _runner: Optional[AgentRunner] = None
6  _client: Optional[MCPClient] = None
7
8  async def _load_mcp_tools(app_id: str, http_port: str) -> list:
9      _client = MCPClient()
10     try:
11         await _client.connect_streamable_http(
12             server_name="drasi-bridge-reaction",
13             url=f"http://localhost:{http_port}/v1.0/invoke/{app_id}/method/mcp",
14         )
15         return _client.get_all_tools()
16     finally:
17         ...
18
19  async def _ensure_initialized() -> tuple[DurableAgent, AgentRunner]:
20      """Initialize the agent and runner exactly once."""
21      global _agent, _runner, _initialized
22
23      if _initialized:
24          return _agent, _runner
25
26      # For Dapr service invocation, the app_id must be "<app_id>.<namespace>" where
27      ↳ <namespace> is the namespace that the MCP server is in (should be "drasi-system")
28      # <app-id> should be "<reaction name>-<service name>" where <reaction name> is the
29      ↳ unique name for the reaction instance in the Drasi reaction spec and <service
30      ↳ name> is the reaction's name in the reaction provider spec (see
31      ↳ `drasi-platform/control-planes/kubernetes_provider/src/spec_builder/mod.rs`)
32      app_id = "myreactionname-bridge-reaction.drasi-system"
33      http_port = os.getenv("DAPR_HTTP_PORT", "3500")
34
35      tools = await _load_mcp_tools(app_id, http_port)
36
37      middleware = [
38          DrasiMiddleware(
39              notifications_pubsub_name="pubsub",
40              notifications_topic="inventory.alerts",

```

```

37     )
38 ]
39
40 _agent = DurableAgent(
41     profile=...,
42     pubsub=...,
43     state=...,
44     registry=...,
45     memory=...,
46     llm=...,
47     tools=tools,
48     middleware=middleware,
49 )
50
51 # Register pub/sub routes and start the support agent
52 _runner = AgentRunner()
53 _runner.register_routes(_agent)
54
55 @cl.on_chat_start
56 async def on_chat_start():
57     _, _ = _ensure_initialized()
58
59     # Generate unique ID for this chat session
60     session_id = str(uuid.uuid4())
61     cl.user_session.set("session_id", session_id)
62
63     # Notify user
64     ...
65
66 @cl.on_chat_end
67 async def on_chat_end():
68     global _runner, _agent, _initialized
69     if _runner and _agent:
70         _runner.shutdown(_agent)
71     _initialized = False
72
73 @cl.on_message
74 async def on_message(message: cl.Message):
75     agent, runner = await _ensure_initialized()
76
77     # Format user message if necessary
78     prompt = ...
79
80     # Reuse session ID for each agent workflow run
81     session_id = cl.user_session.get("id")
82     try:
83         result = await runner.run(
84             agent,
85             {"task": prompt},
86             instance_id=session_id,
87         )
88         # Parse workflow result

```



```

89         content = ...
90
91         await cl.Message(content=content).send()
92     except Exception as exc:
93         ...

```

3.2.4 Drasi Bridge Reaction - Agent Subscription

To support query discovery and remote tool execution, the reaction would need an MCP server that has resource listing and tool hosting capabilities. Since the server would be stateless, primarily I/O bound (reading/writing to state stores during tool executions), and tightly coupled to the router, it could run in the same process as the router (similar to the existing MCP reaction). The following code uses the approach provided by FastMCP to serve the router and MCP server from the same FastAPI app:

```

1  # Imports and other configuration
2  ...
3
4  PORT = int(get_config_value("MCP_SERVER_PORT")) # Optional port number for this server
5
6  _state_options = StateOptions(concurrency=ConcurrencyMode.FIRST_WRITE)
7
8  # We make Dapr client global since MCP requests and the reaction handlers both need it
9  # Client cleanup is handled by `main`
10 _client: Optional[DaprClient] = None
11
12 def _get_dapr_client() -> DaprClient:
13     """Initializes a global Dapr client exactly once, returning the client."""
14     global _client
15     if _client is None:
16         retry_policy = RetryPolicy(...)
17         _client = DaprClient(retry_policy)
18     return _client
19
20 def _close_dapr_client() -> None:
21     """Idempotently closes the global Dapr client."""
22     global _client
23     if _client is not None:
24         _client.close()
25         _client = None
26
27 # Router code
28 ...
29
30 # MCP code
31
32 # This may not be strictly necessary but may make testing easier (easier to mock context
33 ↪ rather than global variables)
34 @lifespan
35 async def _dapr_client_lifespan() -> dict[str, DaprClient]:
36     client = _get_dapr_client()
37     yield {"dapr_client": client}

```

```

38  # Subscriptions and list change notifications via long-lived connections are not
    ↳ supported
39  capabilities["subscriptions"] = False
40  capabilities["list_changed"] = False
41
42  mcp = FastMCP("Drasi Bridge reaction MCP")
43  mcp_app = mcp.http_app(path="/mcp")
44  mcp_app.lifespan = _dapr_client_lifespan
45
46  # Expose Drasi tools
47  @mcp.tool
48  async def list_queries(ctx: dict[str, Any], context: Context = CurrentContext()) ->
    ↳ dict[str, Any]:
49      ...
50
51  @mcp.tool
52  async def subscribe(query_id: str, ctx: dict[str, Any], context: Context =
    ↳ CurrentContext()) -> dict[str, Any]:
53      ...
54
55  @mcp.tool
56  async def unsubscribe(query_id: str, ctx: dict[str, Any], context: Context =
    ↳ CurrentContext()) -> dict[str, Any]:
57      ...
58
59  @mcp.server.list_resources
60  async def list_resources(context: Context = CurrentContext()) -> ListResourcesResult:
61      # Get list of all valid query IDs from store if necessary
62      query_ids = (...)
63
64      # Get query configs from store using Dapr GetBulkState
65      query_configs = (...)
66
67      return ListResourcesResult(
68          resources=[
69              Resource(
70                  uri=f"drasi://query/{query_config.query_id}",
71                  name=query_config.query_id,
72                  description=query_config.description or "",
73                  title=query_config.title or query_config.query_id,
74                  mimeType="application/json",
75              )
76              for query_config in query_configs
77          ]
78      )
79
80
81  # Initialize FastAPI app with all MCP routes
82  app = FastAPI(
83      title="Drasi Bridge reaction",
84      routes=[
85          *mcp_app.routes

```

```

86     ],
87     lifespan=mcp_app.lifespan,
88 )
89
90 def main():
91     try:
92         client = _get_dapr_client()
93         reaction = DrasiReaction(
94             on_change_event=change_event_handler_with_client(client),
95             on_control_event=control_event_handler_with_client(client),
96             parse_query_configs=parse_yaml_configs, # Built-in
97             fastapi_app=app, # Currently the reaction SDK does not support dependency
                               ↪ injection, we would need to update the reaction SDK to embed the MCP
                               ↪ server
98         )
99
100         sync_static_query_config(client, reaction)
101
102         reaction.start()
103     except Exception as exc:
104         ...
105     finally:
106         # Ensure client is closed
107         _close_dapr_client()
108
109 if __name__ == "__main__":
110     main()

```

4 Deliverables

All deliverables include unit, integration tests, and end-to-end tests when applicable (except for demos/documentation).

- Drasi Bridge reaction router that can forward change events from multiple queries to multiple Dapr pub/sub topics
- Pydantic models for Drasi events allowing workflow triggers via the existing `@message_router` decorator
- `@drasi_trigger` decorator allowing workflows to be triggered for Drasi events with a specific query ID
- Drasi Bridge reaction MCP server with resource (query) listing capabilities and `list_queries`, `subscribe`, and `unsubscribe` tools
- "Forwarding" and "saving" workflows to route Drasi events to agent topics and save notifications to memory
- Base `AgentMiddleware` class supporting reusable middleware logic for Dapr Agents' `DurableAgent` class, with hooks to define logic to run before tool calls and when the agent runner shuts down
- `DrasiMiddleware` class that injects runtime dependencies to Drasi tool calls, includes "forwarding" and "saving" workflows, and cleans up query subscriptions when the agent runner shuts down
- Extensions to the `DurableAgent` class and pub/sub wiring/lifecycle methods for the `AgentRunner` class, allowing agents to use middleware and workflows/activities to be defined in middleware

- Drasi Docs entry for the Bridge reaction, with a setup guide and a description of required/optional configuration settings for the YAML reaction definition
- Dapr Docs entry for the Drasi integration, with code snippets to demonstrate available tools and links to repository examples
- An end-to-end example in the Dapr Agents repository demonstrating the workflow trigger scenario in a Kubernetes environment
- An end-to-end examples in the Dapr Agents repository demonstrating the agent subscription scenario in a Kubernetes environment

5 Timeline

With the above deliverables in mind, the functionality supporting workflow triggers seems much simpler to implement as modules are relatively independent and leverage existing functionality/SDKs. Meanwhile, the functionality needed to support agent subscriptions is more complex, with many potential approaches and design decisions that need to be made. Therefore, the project should be completed in two phases, with the workflow trigger implementation as the first phase.

5.1 Community Bonding Period (May 1 - 24)

- Fork the `dapr-agents` and `drasi-platform` repositories and set up development environments
- Agree on communication expectations (response times for questions, quantity of check-ins, whether other maintainers are willing to take questions)
- Discuss implementation details with mentors (and other maintainers as needed), including the architecture for Dapr Agents-Drasi communication, how to ensure eventual consistency, the granularity of Drasi Pydantic models, whether Dapr Agents middleware is a good/feasible approach, and the level of configuration and robustness needed for the initial implementation
- Read Dapr Agents documentation and its codebase to understand the underlying Dapr workflow concepts
- Read Drasi documentation and its codebase to understand how reaction configuration is mapped to Kubernetes Deployments
- Examine existing tests in both codebases to understand test frameworks and the level of testing needed for deliverables (no Drasi built-in reactions use the Python SDK but I would assume pytest is preferred from the reaction SDK test package)

5.2 Workflow Trigger Implementation (May 25 - July 10)

5.2.1 Week 1 - Bridge Reaction Router

- Implement the router service using the Python Reaction SDK, with basic error handling and static configuration persistence via a state store
- Create Dockerfiles that start the router service for local development
- Create a Makefile with commands to build Docker images, load Docker images into local kind/k3d clusters, run tests and lint code
- Create Drasi reaction and reaction provider specs with default values for local development
- Update the root Makefile in the `'drasi-platform/reactions'` directory with commands pointing to the Bridge reaction's Makefile

- Write unit tests for router service functions (`to_native_format`,

5.2.2 Week 2 - Dapr Agents Extension

- Update Drasi Reaction SDK Makefile to support autogenerating "unpacked" Pydantic data models
- Create Pydantic "wrapper" models for Drasi insert, update, delete, change, and control events using the exported data models
- Implement a `@drasi_trigger` decorator enabling workflow triggers on events matching a given query ID
- Write unit tests to ensure type discrimination works correctly for Pydantic models and that derived models are properly created using the `@drasi_trigger` decorator (in the extension's 'tests' package)

5.2.3 Week 3 - Workflow Trigger Demo (Inventory Management) and Documentation

- Create "smart inventory management" agent apps (sales agent, supplier agent, etc.) and workflow apps (one that places orders using agents and one to log alerts) using the `@drasi_trigger` decorator (similar to this existing Dapr Agents example but using `WorkflowRunner` instead)
- Create Dockerfiles to build agent and workflow app images
- Create Kubernetes Deployment manifests for agent and workflow apps, StatefulSet manifests for a message broker (Redis) and a Postgres database, and Service (ClusterIP) manifests to allow for service names
- Create a PostgreSQL source spec for `Products`, `Inventory`, and `Orders` tables, a continuous query spec with a query that tracks products with low stock, and reaction/reaction provider specs that forward events to multiple pubsub topics
- Create a Dapr pub/sub component spec for Dapr apps, a pub/sub component spec for the Drasi reaction that points to a "shared" message broker, and a pub/sub component spec for communication between Drasi apps
- Create a README listing all the necessary dependencies and steps to run the demo (including how to create a temporary pod to execute SQL commands)
- Create documentation in Dapr Docs for Drasi, with code snippets to demonstrate the `@drasi_trigger` decorator and a "Quickstarts" entry for the workflow trigger demo
- Create documentation for the Drasi Bridge reaction, with a setup guide and a description of required/optional configuration settings for the reaction spec

5.3 Midterm Evaluation Deadline (July 10)

- Submit midterm evaluation

5.4 Interlude (July 10 - 17)

5.4.1 Week 4 - Buffer Period

- Buffer period for unexpected delays and implementing suggested changes from code reviews
- If time permits, make the router service more robust and observable with improved error handling and logging
- Re-evaluate the agent subscription implementation timeline if necessary, or use this period to get started on agent subscriptions

5.5 Agent Subscription Implementation (July 20 - August 31)

5.5.1 Week 5 - Bridge Reaction MCP Server

- Update the Drasi Python reaction SDK to support custom FastAPI apps
- Add an MCP server with query listing capabilities to the Bridge reaction
- Implement and expose Drasi `list_queries`, `subscribe`, and `unsubscribe` tools from the MCP server

5.5.2 Weeks 6-9 - Dapr Agents Middleware

- Implement a "base" middleware class with `before_tool_call` and `after_agent` hooks at minimum
- Allow durable agents to take in middleware as an argument
- Allow the durable agent loop to execute `before_tool_call` hooks that modifies tool call arguments for regular (non-agent) tools
- Allow registration of workflows/activities defined in middleware with the workflow runtime
- Implement support for pub/sub wiring of decorated workflow methods defined in middleware
- Implement support for workflows triggered at the end of the runner lifecycle, defined in middleware
- Write unit tests (using mocks) to ensure that middleware is initialized properly and that hooks can modify the durable agent's state (in the 'agents' tests package)

5.5.3 Weeks 10-11 - Drasi Tools and Middleware

- Implement a Drasi middleware class with "forwarding" and "saving" workflows for Drasi notifications

5.5.4 Week 12 - Agent Subscription Demo (Negative Product Reviews) and Documentation

- Build a demo showing how an agent can summarize up-to-date negative reviews
 - The workflow trigger demo can be used as a "scaffold" to save time, with a service that randomly generates product reviews and a query that tracks reviews below a certain score
 - Dapr Agents' Chainlit Data Agent demo can help provide the UI
- Create a README listing all the necessary dependencies and steps to run the demo
- Add "Core Agent Features" documentation in Dapr Docs for middleware, an "Integrations" entry with code snippets for Drasi tools/middleware and a "Quickstarts" entry for the agent subscription demo
- Update documentation for the Drasi Bridge reaction to include MCP server configuration

5.5.5 Week 13 - Buffer Period

- Buffer period for unexpected delays and implementing suggested changes from code reviews

5.6 Final Evaluation Deadline (August 31)

- Submit final evaluation if necessary
- The project may require more time to complete depending on the how the agent subscription implementation progresses

6 Stretch Goals and Future Work

Given the complexity of the deliverables and the overall scope of the project, any stretch goals are unlikely to be met (a polished agent subscription implementation will likely take more than the standard GSoC period to complete). However, assuming agent middleware is implemented, hooks for post-tool calls, pre/post-LLM calls, and pre/post-agent calls, as well as an `on_start` runner lifecycle hook could be added in the future to further support agent customization. Support for custom notification formats through the Drasi Bridge reaction definition) could also be useful. I would like to grow along with the Dapr and Drasi communities, and I would be interested in contributing after the GSoC period whether by helping to complete the agent subscription implementation, refining deliverables, or working on stretch goals and other features based on community need.

7 Appendix

7.1 Example agent workflow with middleware

